Zaucha, Wojciech, 2420959
Nagy Edvárd 2423665

IOT project report

Metropolia University of Applied Sciences

Internet of things

09.03.2026.

1. Introduction

The purpose of this project was to build a simple IoT system that collects temperature data, processes it, stores it in databases, and displays the latest value on a dashboard. The system uses MQTT communication, a Raspberry Pi as the processing node, and multiple databases to store different types of information.

Several technologies were used to implement the system. These include MQTT with a HiveMQ Cloud broker for communication, Python for data processing, MongoDB for storing historical sensor data, Redis for storing the latest temperature value, PostgreSQL for storing device metadata, and Flask for creating a simple web dashboard. The system was implemented using the following technologies:

- ESP32 used as a sensor simulator
- MQTT protocol with HiveMQ Cloud broker
- Python data processing pipeline script
- MongoDB database for historical data storage
- Redis database for real-time data caching
- PostgreSQL database for device metadata
- Flask web server for dashboard visualization
- Raspberry Pi as the edge processing node

In this project, the ESP32 simulates a temperature sensor by generating temperature values and sending them using the MQTT protocol. These generated sensor data also average filtered locally inside the ESP32. These messages are then received by a Python script running on the Raspberry Pi.

2. Data Processing and Storage

When the Raspberry Pi receives a temperature message from the MQTT broker, the Python script processes the data and stores it in multiple databases depending on its purpose.

MongoDB is used to store the full history of temperature readings. Each measurement is saved together with the device ID and timestamp. This allows the system to keep a complete record of all sensor data, which could later be used for analysis.

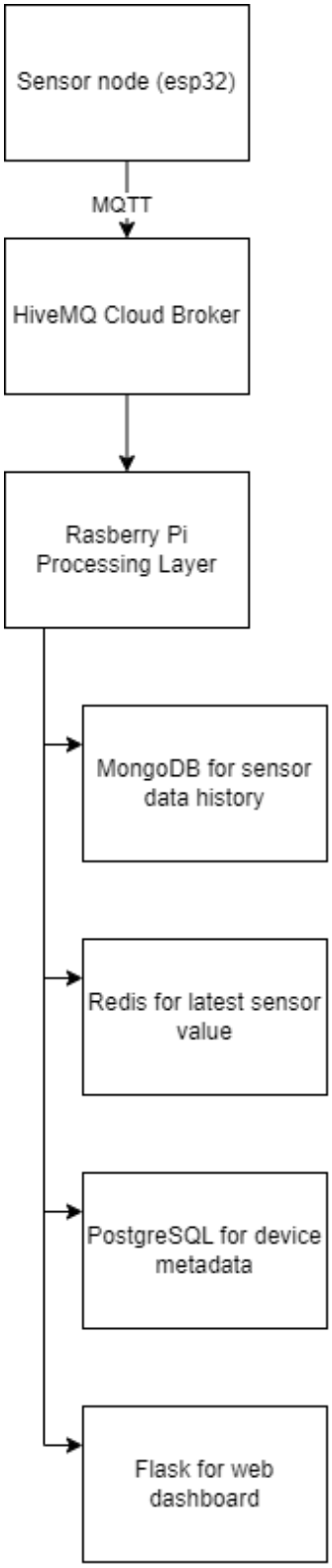
Redis is used as a fast in-memory database that stores only the latest temperature value. This makes it very efficient for real-time applications such as dashboards, because the newest value can be retrieved immediately without searching through large datasets.

PostgreSQL is used to store device-related information such as device ID, sensor type, and location. This type of structured information is well suited for relational databases.

3. System Architecture

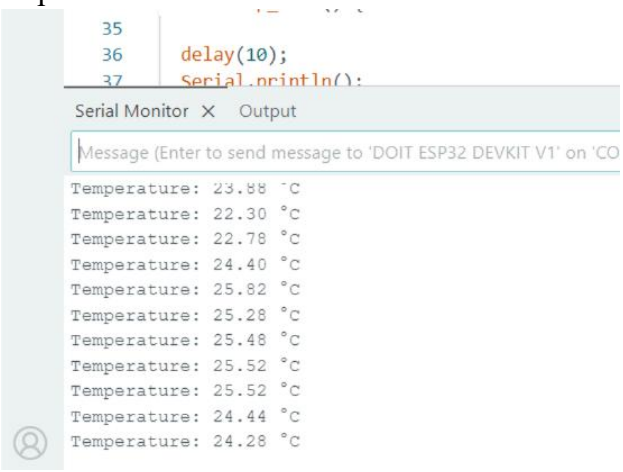
The implemented IoT system follows a layered architecture that is commonly used in industrial IoT systems.

The overall system architecture is illustrated conceptually below:

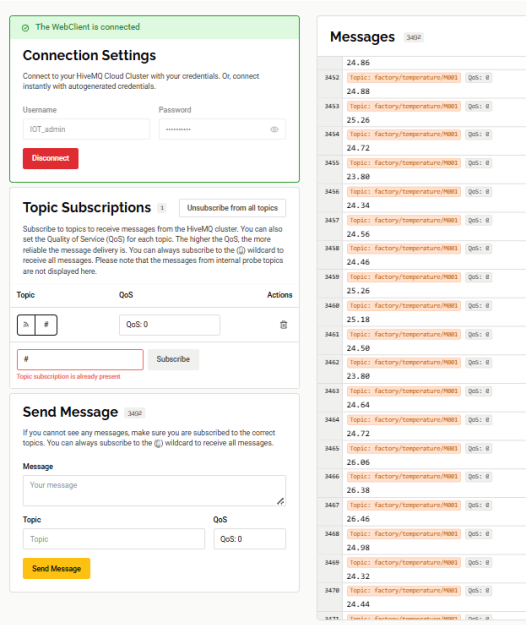


3. Screenshots of the implementations

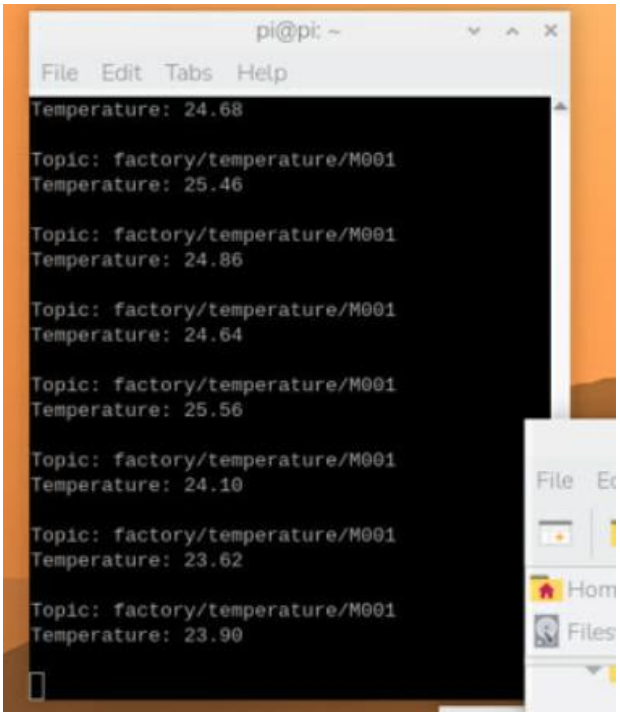
Esp32 serial monitor:



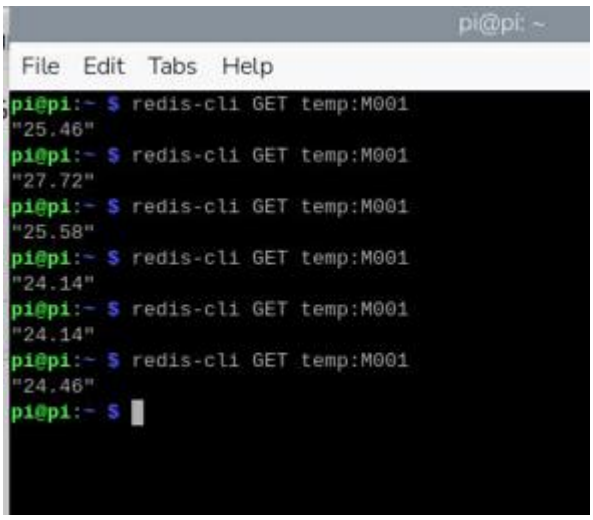
HiveMQ MQTT:



Pi MQTT receive:



Redis:



MongoDB:

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&
File  Edit  Tabs  Help
},
{
  _id: ObjectId('69adc08830d408a2dc0e421a'),
  device: 'M001',
  temperature: 24.1,
  timestamp: ISODate('2026-03-08T20:31:36.659Z')
},
{
  _id: ObjectId('69adc08a30d408a2dc0e421b'),
  device: 'M001',
  temperature: 23.48,
  timestamp: ISODate('2026-03-08T20:31:38.664Z')
},
{
  _id: ObjectId('69adc08c30d408a2dc0e421c'),
  device: 'M001',
  temperature: 24.5,
  timestamp: ISODate('2026-03-08T20:31:40.680Z')
},
{
  _id: ObjectId('69adc08e30d408a2dc0e421d'),
  device: 'M001',
  temperature: 25.48,
  timestamp: ISODate('2026-03-08T20:31:42.676Z')
},
{
  _id: ObjectId('69adc09030d408a2dc0e421e'),
  device: 'M001',
  temperature: 26.38,
  timestamp: ISODate('2026-03-08T20:31:44.664Z')
```

MongoDB:

```
File  Edit  Tabs  Help
Temperature: 25.42
Timestamp: 2026-03-08 21:41:17.186550
Redis updated: temp:M001 = 25.42
MongoDB document inserted
----------------------------------
Device: M001
Temperature: 26.28
Timestamp: 2026-03-08 21:41:19.308303
Redis updated: temp:M001 = 26.28
MongoDB document inserted
----------------------------------
Device: M001
Temperature: 27.06
Timestamp: 2026-03-08 21:41:21.115940
Redis updated: temp:M001 = 27.06
MongoDB document inserted
----------------------------------
Device: M001
Temperature: 27.78
Timestamp: 2026-03-08 21:41:23.293618
Redis updated: temp:M001 = 27.78
MongoDB document inserted
----------------------------------
Device: M001
Temperature: 27.78
Timestamp: 2026-03-08 21:41:25.188791
Redis updated: temp:M001 = 27.78
MongoDB document inserted
----------------------------------
Device: M001
Temperature: 27.4
Timestamp: 2026-03-08 21:41:27.296893
Redis updated: temp:M001 = 27.4
MongoDB document inserted
```

MongoDB:

```
pi@pi: ~
File  Edit  Tabs  Help
Saved to MongoDB: {'device': 'M001', 'temperature': 23.88, 'timestamp': datetime.
40, 27, 104883), '_id': ObjectId('69add0ab30d408a2dc0e4a2a')}
Saved to MongoDB: {'device': 'M001', 'temperature': 24.7, 'timestamp': datetime.d
0, 29, 93610), '_id': ObjectId('69add0ad30d408a2dc0e4a2b')}
Saved to MongoDB: {'device': 'M001', 'temperature': 26.22, 'timestamp': datetime.
40, 31, 100926), '_id': ObjectId('69add0af30d408a2dc0e4a2c')}
Saved to MongoDB: {'device': 'M001', 'temperature': 26.28, 'timestamp': datetime.
40, 33, 221389), '_id': ObjectId('69add0b130d408a2dc0e4a2d')}
Saved to MongoDB: {'device': 'M001', 'temperature': 25.82, 'timestamp': datetime.
40, 35, 103258), '_id': ObjectId('69add0b330d408a2dc0e4a2e')}
Saved to MongoDB: {'device': 'M001', 'temperature': 26.78, 'timestamp': datetime.
40, 37, 215015), '_id': ObjectId('69add0b530d408a2dc0e4a2f')}
Saved to MongoDB: {'device': 'M001', 'temperature': 26.34, 'timestamp': datetime.
40, 39, 96939), '_id': ObjectId('69add0b730d408a2dc0e4a30')}
Saved to MongoDB: {'device': 'M001', 'temperature': 25.9, 'timestamp': datetime.d
0, 41, 212551), '_id': ObjectId('69add0b930d408a2dc0e4a31')}
Saved to MongoDB: {'device': 'M001', 'temperature': 25.76, 'timestamp': datetime.
40, 43, 101899), '_id': ObjectId('69add0bb30d408a2dc0e4a32')}
Saved to MongoDB: {'device': 'M001', 'temperature': 27.26, 'timestamp': datetime.
40, 45, 341892), '_id': ObjectId('69add0bd30d408a2dc0e4a33')}
Saved to MongoDB: {'device': 'M001', 'temperature': 26.78, 'timestamp': datetime.
40, 47, 261838), '_id': ObjectId('69add0bf30d408a2dc0e4a34')}
Saved to MongoDB: {'device': 'M001', 'temperature': 27.3, 'timestamp': datetime.d
0, 49, 182377), '_id': ObjectId('69add0c130d408a2dc0e4a35')}
```

PostgreSQL:

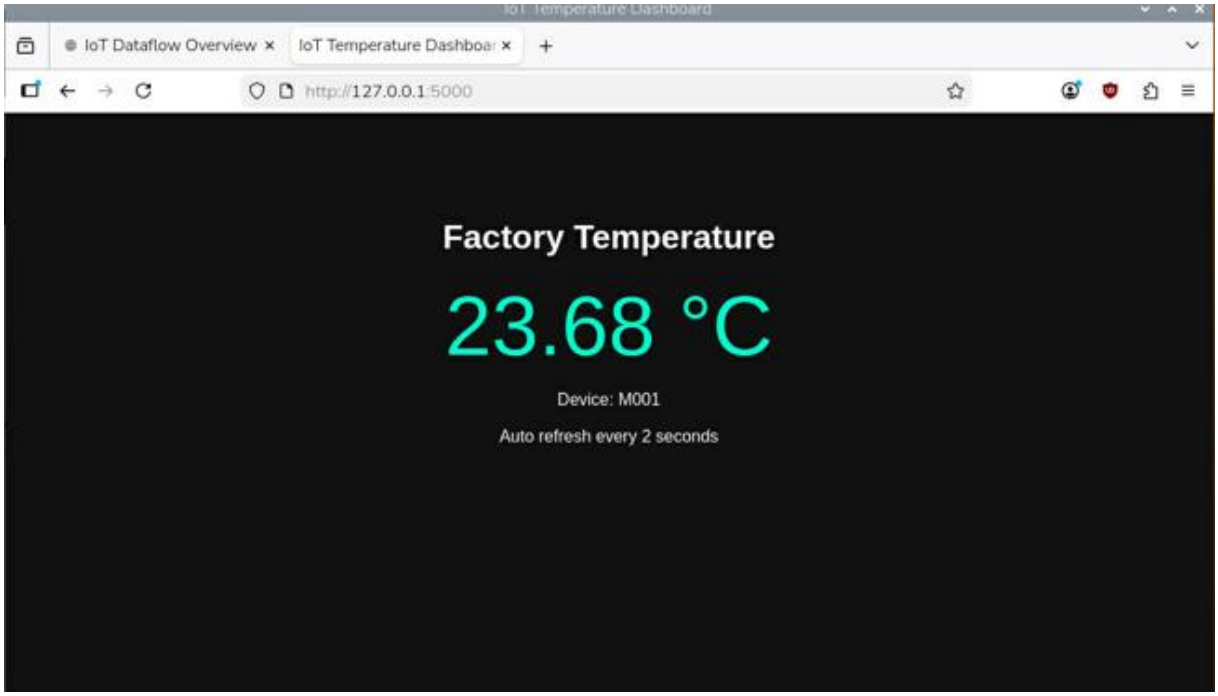
```
pi@pi: ~
File  Edit  Tabs  Help
iot=# CREATE TABLE devices (
device_id TEXT PRIMARY KEY,
location TEXT,
sensor_type TEXT
);
CREATE TABLE
iot=# INSERT INTO devices VALUES (
'M001',
'Factory Room A',
'Temperature Sensor'
);
INSERT 0 1
iot=# SELECT * FROM devices;
 device_id |     location     |     sensor_type
-----------+------------------+--------------------
 M001      | Factory Room A | Temperature Sensor
(1 row)

iot=# SELECT * FROM devices;
 device_id |     location     |     sensor_type
-----------+------------------+--------------------
 M001      | Factory Room A | Temperature Sensor
(1 row)

iot=# SELECT * FROM devices;
 device_id |     location     |     sensor_type
-----------+------------------+--------------------
 M001      | Factory Room A | Temperature Sensor
(1 row)

iot=#
```

Flask dashboard:



4. Challenges and difficulties

During the implementation of the system several challenges were encountered. One of the main difficulties was configuring the different databases and ensuring that each service was running properly on the Raspberry Pi. Because the system uses multiple technologies such as MongoDB, Redis, PostgreSQL, and MQTT, it was important to make sure that all components could communicate correctly with the Python processing script.

Another challenge was working with the different command environments. At times, commands were accidentally executed in the wrong context, such as running Python code in the Linux terminal or executing database commands outside their correct shells. Understanding when to use the Linux terminal, MongoDB shell, Redis CLI, or Python interpreter required some troubleshooting and practice.

5. Conclusion

This project demonstrates a complete IoT data pipeline from sensor data generation to real-time visualization. The system shows how different databases can be used for different purposes within real IoT architecture. MongoDB is used for storing historical sensor data, Redis is used for fast access to the latest measurement, and PostgreSQL stores structured device information.

The final system successfully receives temperature data through MQTT, processes it on a Raspberry Pi, stores the data in multiple databases, and displays the most recent value on a web-based dashboard. This implementation provides a clear example of how modern IoT systems handle data collection, storage, and visualization.